

《计算机网络》实验报告

实验 1：利用流式套接字编写聊天程序

姓名：蒋衲言 学号：2313546

1 实验要求

1. 设计聊天协议，并给出聊天协议的完整说明。
2. 利用 C 或 C++ 语言，使用基本的 Socket 函数进行程序编写，不允许使用 CSocket 等封装后的类。
3. 程序应有基本的对话界面，但可以不是图形界面。程序应有正常的退出方式。
4. 完成的程序应能支持英文和中文聊天。
5. 采用多线程，支持多人聊天。
6. 编写的程序应结构清晰，具有较好的可读性。
7. 在实验中观察是否有数据包的丢失，提交程序源码、可执行代码和实验报告。

2 协议设计

在我编写的聊天软件中，每条消息统一采用以下通用格式：

< 类型 >:< 发送者 >:< 内容长度 >:< 内容 >

字段说明如下：

字段名	类型	示例	含义
类型	字符串	JOIN/MSG/LEAVE	表示消息类型（固定 5 字节）
发送者	字符串	jry/系统	表示消息来源（最大 32 字节）
内容长度	4 位十进制数	0000/0024	表示消息内容的长度
内容	字符串	"欢迎加入聊天室！"	实际消息内容，可包含中英文字符

三种消息类型的具体说明如下：

1. JOIN 消息

格式：

JOIN:< 用户名 >:0000:

当客户端第一次连接并发送用户名后,服务器广播一条该格式的消息给其他所有用户。JOIN 消息的内容长度固定为 0000, 因为 JOIN 消息没有正文内容。客户端接收到此消息后, 会输出类似:

```
>>> 系统: 用户 [jry] 加入了聊天室
```

对应代码如下:

server.c

```
sprintf_s(message, BUFFER_SIZE + 100, "JOIN:%s:0000:", client->name);  
broadcast_message(message, client->socket);
```

2. MSG 消息

格式:

```
MSG:< 用户名 >:< 长度 >:< 消息内容 >
```

MSG 消息代表普通聊天消息, 由服务器转发(广播)给除发送者外的所有客户端。内容可以是中文、英文或混合文本。例如:

```
MSG:Alice:0008: 大家好!
```

```
MSG:Bob:0011:Hello world
```

特别注意: 在 UTF-8 编码下, 英文字符(ASCII)占 1 个字节, 中文字符占 3 个字节。但是 Visual Studio 默认不使用 UTF-8 编码, 而是使用 **ANSI 编码**(也就是系统本地代码页, Windows 中国大陆通常是 GBK/CP936)来保存源文件里的中文字符串。在 GBK 编码中, ASCII 字符占 1 字节(和 UTF-8 相同), **中文字符占 2 字节**。

对应代码如下:

server.c

```
sprintf_s(message, BUFFER_SIZE + 100, "MSG:%s:%04d:%s", client->name,  
    (int)strlen(buffer), buffer);  
broadcast_message(message, client->socket);
```

客户端解析部分:

client.c

```
char* type = strtok(buffer, ":");  
char* sender = strtok(NULL, ":");  
char* length_str = strtok(NULL, ":");  
char* content = strtok(NULL, "");
```

然后根据类型判断打印输出。

3. LEAVE 消息

格式:

```
LEAVE:< 用户名 >:0000:
```

当用户主动输入`exit`或断开连接时,服务器会广播此消息。客户端接收到后打印提示:

```
>>> 系统: 用户 [jry] 离开了聊天室
```

对应代码如下:

server.c

```
sprintf_s(message, BUFFER_SIZE + 100, "LEAVE:%s:0000:", client->name);  
broadcast_message(message, client->socket);
```

3 程序编写

3.1 Windows Socket 编程简述

套接字有流式套接字 (SOCK_STREAM)、数据报套接字 (SOCK_DGRAM) 等类型, TCP 用 SOCK_STREAM (有连接、可靠), UDP 用 SOCK_DGRAM (无连接、非可靠、保留消息边界)。本次实验的聊天程序当然用的是 SOCK_STREAM。

在 Windows 操作系统下, 利用 TCP 服务的应用程序编写步骤如下:

服务器端

```
WSAStartup() → socket() → bind() → listen() → accept() → send()/recv()  
→ closesocket() → WSACleanup()
```

客户端

```
WSAStartup() → socket() → connect() → send()/recv() → closesocket()  
→ WSACleanup()
```

相较于 Linux 系统, 在 Windows 系统上编写 Socket 程序有一些需要单独注意的点:

- 必须包含的头文件`#include <winsock2.h>`, 它包含 Windows Socket API (Winsock) 提供的核心接口。
- 必须有`#pragma comment(lib, "ws2_32.lib")`, 它告诉编译器在链接阶段时自动链接`ws2_32.lib`这个静态库。
- 必须初始化与清理 Winsock, 即`WSAStartup()`需要在任何`socket()`调用之前执行, `WSACleanup()`必须在程序退出前调用。

3.2 服务器端 server.c

server.c 要实现一个支持多人同时聊天的服务器端程序，功能包括：

1. 启动监听端口，等待客户端连接。
2. 每当有客户端连接，就创建一个线程处理该客户端的消息。
3. 当某个客户端发来消息时，服务器将该消息广播给所有其他客户端。
4. 当客户端退出时，服务器正确关闭其套接字，释放资源。

此处仅对 server.c 的关键地方作说明，完整程序详见 C 源文件。

3.2.1 创建监听套接字

server.c

```
SOCKET server_socket;
server_socket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
if (server_socket == INVALID_SOCKET) {
    printf("Socket 创建失败: %ld\n", WSAGetLastError());
    WSACleanup();
    return 1;
}
```

server_socket并不直接收发数据，它的作用是监听来自客户端的连接请求。AF_INET表示使用 IPv4 地址族，SOCK_STREAM表示使用流式套接字，IPPROTO_TCP表示使用 TCP 协议。

3.2.2 绑定本地 IP 地址和端口

server.c

```
struct sockaddr_in server_addr;

server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(port);

if (bind(server_socket, (struct sockaddr*)&server_addr, sizeof(
server_addr)) == SOCKET_ERROR) {
    printf("绑定失败: %d\n", WSAGetLastError());
    closesocket(server_socket);
    WSACleanup();
    return 1;
}
```

此处为服务器套接字指定通信协议(IPv4)和地址(IP+ 端口),并将该地址与 socket 绑定。其中INADDR_ANY是一个特殊的常量宏,定义为#define INADDR_ANY ((unsigned long)0x00000000),也就是说它代表 IPv4 地址 0.0.0.0,这是一个通配地址,让服务器绑定到本机的所有可用网络接口上。

3.2.3 启动监听

server.c

```
if (listen(server_socket, SOMAXCONN) == SOCKET_ERROR) {
    printf("监听失败: %d\n", WSAGetLastError());
    closesocket(server_socket);
    WSACleanup();
    return 1;
}
```

调用listen()函数,表示服务器现在可以接受连接请求。第二个参数SOMAXCONN表示允许系统决定最大等待连接数。

3.2.4 等待客户端连接

server.c

```
SOCKET client_socket;
struct sockaddr_in client_addr;
int client_addr_len = sizeof(client_addr);

while (1) {
    client_socket = accept(server_socket, (SOCKADDR*)&client_addr, &
        client_addr_len);
    // ...
}
```

此处又创建了一个 socket, client_socket。当某个客户端执行connect(server_socket, ...)时,服务器端调用client_socket = accept(server_socket, ...),表示一个已经建立的 TCP 连接,是专属于某个客户端的通信通道。accept()是一个阻塞函数,直到有客户端连接才返回。简而言之, server_socket专用于监听, client_socket专用于通信,因此服务器需要两个套接字变量。

3.2.5 为每个客户端创建线程

server.c

```
HANDLE thread_handle = (HANDLE)_beginthreadex(NULL, 0, handle_client,
    (void*)client, 0, NULL);
if (thread_handle == NULL) {
    printf("线程创建失败\n");
    closesocket(client_socket);
    //free(client); // remove_client里已包含free, 不能重复释放
    remove_client(client_socket);
}
else {
```

```
    CloseHandle(thread_handle);  
}
```

这里创建一个独立线程，专门处理该客户端的收发消息。函数_beginthreadex()的作用是：在 Windows 上创建一个新的线程，并正确初始化 C 运行时库（CRT, C Runtime）环境。相较于CreateThread()函数，_beginthreadex()更安全，不会内存泄漏。因为我的线程函数handle_client中调用了printf()、free()等依赖于 C 运行时库的线程局部存储的函数，而CreateThread()不会为线程初始化这些内容，所以这类函数可能会出现内存泄漏、异常退出等情况。需要注意的是，_beginthreadex()的返回值类型是uintptr_t而不是HANDLE，所以需要强制类型转换一下。

函数_beginthreadex()的原型如下：

```
#include <process.h>  
uintptr_t _beginthreadex(  
    void *security, // 安全属性，一般为 NULL  
    unsigned stack_size, // 线程栈大小 (0 表示默认)  
    unsigned (__stdcall *start_address)(void *), // 线程函数  
    void *arglist, // 传递给线程函数的参数  
    unsigned initflag, // 0表示立刻运行，CREATE_SUSPENDED表示挂起  
    unsigned *thrdaddr // 可选，返回线程ID  
);
```

最后CloseHandle()函数用于关闭一个由 Windows API 创建的内核对象句柄，释放系统资源。

3.2.6 接收客户端消息并广播

```
server.c  
  
recv_len = recv(client->socket, buffer, BUFFER_SIZE - 1, 0);  
  
if (recv_len == SOCKET_ERROR) {  
    int error_code = WSAGetLastError();  
    if (error_code != WSAEWOULDBLOCK) {  
        printf("接收错误: %d\n", error_code);  
        error = 1;  
    }  
    break;  
}  
  
if (recv_len == 0) {  
    printf("客户端 %s 主动断开连接\n", client->name);  
    break;  
}
```

```

buffer[recv_len] = '\0';
trim_newline(buffer);

if (strcmp(buffer, "exit") == 0) {
    printf("用户 %s 请求退出\n", client->name);
    break;
}

if (strlen(buffer) == 0) {
    continue;
}

```

这里调用 `recv()` 函数接收客户端发送的消息内容。

server.c

```

sprintf_s(message, BUFFER_SIZE + 100, "MSG:%s:%04d:%s", client->name,
        (int)strlen(buffer), buffer);
printf("转发消息 [%s]: %s\n", client->name, buffer);
broadcast_message(message, client->socket);

void broadcast_message(char* message, SOCKET sender_sock) {
    WaitForSingleObject(clients_mutex, INFINITE);

    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (clients[i] && clients[i]->socket != sender_sock) {
            if (send(clients[i]->socket, message, strlen(message), 0)
                == SOCKET_ERROR) {
                printf("发送失败到客户端 %s\n", clients[i]->name);
            }
        }
    }

    ReleaseMutex(clients_mutex);
}

```

紧接着对消息内容格式化，转换成自定义的<类型>:<发送者>:<内容长度>:<内容>标准格式并封装进message，接着调用**broadcast_message()**函数进行广播。**broadcast_message()**函数中调用了**send()**函数发送封装好的消息。

3.2.7 互斥锁的设计

server.c

```
HANDLE clients_mutex;
clients_mutex = CreateMutex(NULL, FALSE, NULL);
// ...
WaitForSingleObject(clients_mutex, INFINITE);
// ...
ReleaseMutex(clients_mutex);
```

首先通过CreateMutex()创建了一个 Windows 内核级互斥量clients_mutex, 用来实现多线程之间的互斥访问。WaitForSingleObject()函数表示当前线程申请互斥锁, 如果其他线程已经拿到锁, 这个线程就会阻塞等待。ReleaseMutex()函数释放锁, 允许其他线程进入关键区。

在我的服务器中, 每当一个客户端连接时, 都会创建一个新线程, 每个线程都会调用add_client()、remove_client()、broadcast_message()等函数, 这些函数都访问或修改了同一个全局资源client_t* clients[MAX_CLIENTS]和int client_count。如果没有互斥锁, 多个线程同时修改这些共享数据, 会引发竞争。

3.3 客户端 client.c

client.c 实现的是聊天程序的客户端, 它要完成以下任务:

1. 初始化 Windows Socket 库。
2. 创建套接字, 并连接到服务器。
3. 启动一个线程, 用于接收服务器发来的广播消息。
4. 主线程负责从键盘读取输入并发送消息。
5. 当用户输入“exit”或断开时, 关闭套接字并退出。

client.c 的创建客户端套接字socket()、连接服务器connect()、创建接收线程_beginthreadex()等关键部分的编写和 server.c 非常类似, 因此不再赘述。此处仅对 client.c 的其他关键地方作说明, 完整程序详见 C 源文件。

3.3.1 解析协议消息

client.c

```
recv_len = recv(sock, buffer, BUFFER_SIZE - 1, 0);

buffer[recv_len] = '\0';

char* type = strtok(buffer, ":");
char* sender = strtok(NULL, ":");
char* length_str = strtok(NULL, ":");
char* content = strtok(NULL, "");

if (type && sender) {
```



```

if (strcmp(type, "JOIN") == 0) {
    printf("\n>>> 系统: 用户 [%s] 加入了聊天室\n", sender);
}
else if (strcmp(type, "LEAVE") == 0) {
    printf("\n>>> 系统: 用户 [%s] 离开了聊天室\n", sender);
}
else if (strcmp(type, "MSG") == 0) {
    if (strcmp(sender, "系统") == 0) {
        printf("\n>>> %s\n", content);
    }
    else {
        printf("\n[%s]: %s\n", sender, content);
    }
}
}
else {
    printf("\n>>> 收到无法解析的消息: %s\n", buffer);
}
}

```

调用recv()函数将消息读进了buffer，再由strtok()函数分割。接着通过 type 和 sender 进行一些判断，系统发的消息由“>>>”开头，其他客户端发的消息由用户名开头。strtok()函数是 C 标准库 string.h 中的一个字符串分割函数，作用是把一个字符串按指定的分隔符切成若干片段。函数原型如下：

```

#include <string.h>

// 返回指向下一个“分割出的子串”的指针（类型 char*）。
// 如果没有更多的子串可分割，返回NULL。
char *strtok(
    char *str, // 要分割的原始字符串（第一次调用时传入字符串本身；之后传入
               NULL继续上一次分割）
    const char *delim // 一个字符串，包含所有分隔符字符（例如":"或",";")
);

```

注意：strtok()会在每次找到一个分隔符时，把它替换成'\0'，所以strtok()会修改原字符串。

3.3.2 输入用户名的检测

```

client.c

#define MAX_NAME_LEN 32

while (1) {
    printf("请输入你的用户名（最多%d字节）: ", MAX_NAME_LEN - 2);
    if (fgets(username, MAX_NAME_LEN, stdin) == NULL) {

```

```

    printf("读取用户名失败\n");
    WSACleanup();
    return 1;
}

size_t len = strlen(username);

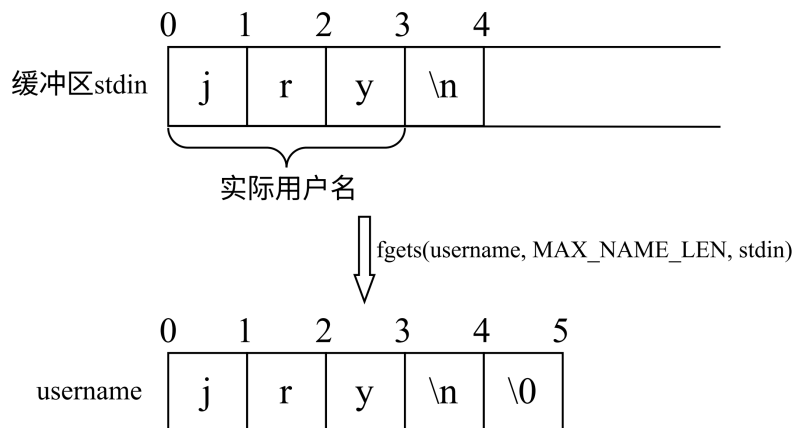
// 如果最后一个字符是换行则去掉
if (len > 0 && username[len - 1] == '\n') {
    trim_newline(username);
    if (strlen(username) == 0) {
        printf("用户名不能为空，请重新输入.\n");
        continue;
    }
    break; // 合法输入
}
else {
    // 未读到换行，说明输入超出缓冲区（或输入恰好填满缓冲区）
    int ch;
    while ((ch = getchar()) != '\n' && ch != EOF) { } // 清理stdin
        中剩余的字符直到遇到换行或EOF（重要，不能删除这段代码！）

    printf("用户名不能超过 %d 字节，请重新输入.\n", MAX_NAME_LEN
        - 1);
    continue;
}
}

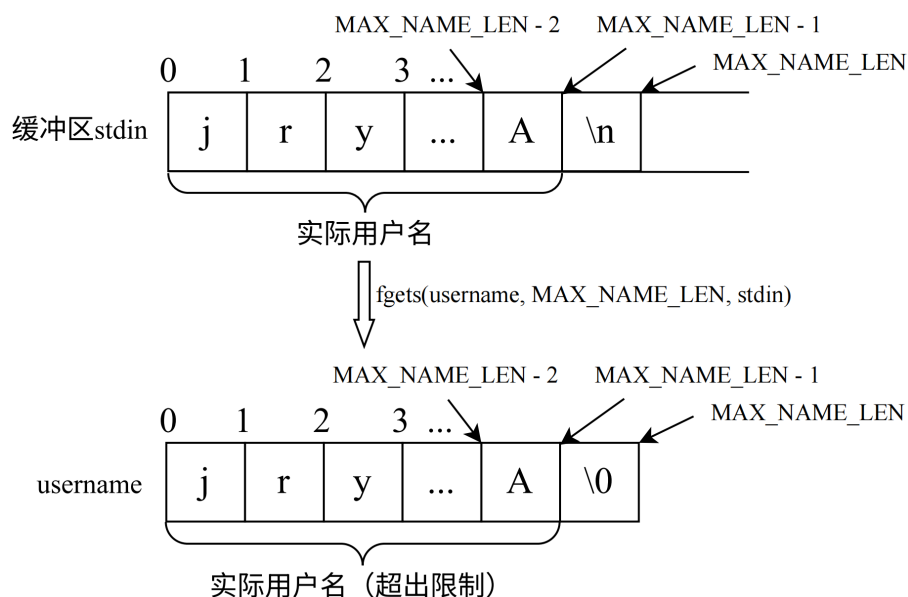
```

`fgets()`函数从用户输入`stdin`读取内容，`fgets()`会在读到换行符`\n`、读到文件结束（EOF）或者读满`MAX_NAME_LEN - 1`个字节三者之一时停止读取，并用`'\0'`终止。可能有以下几种情况：

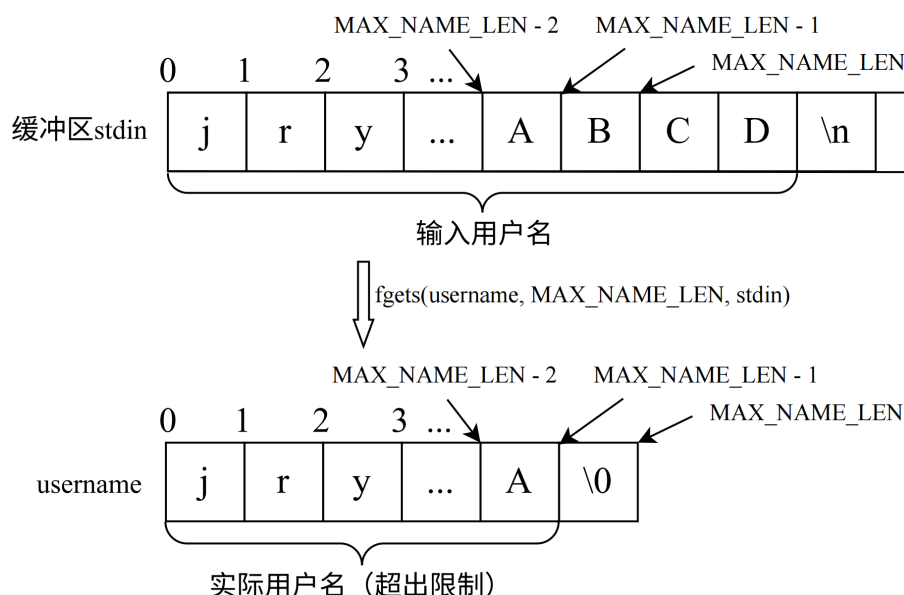
1. 输入的实际用户名长度小于`MAX_NAME_LEN - 1`个字节：



2. 输入的实际用户名长度等于`MAX_NAME_LEN - 1`个字节：



3. 输入的实际用户名长度大于MAX_NAME_LEN - 1个字节:



并且当用户输入用户名时，fgets()会保留换行符\n（如果长度装得下的话），strlen()函数返回的是字符串不含'\0'的长度，所以可以根据username[strlen(username) - 1]是否是换行符\n来判断输入的用户名有没有超出限制。

当输入过长时，fgets()只读取前一部分，剩下的字符（包括换行符）还留在输入缓冲区里。因此必须手动清空缓冲区，否则下一次输入会直接读取旧数据。这就是getchar()函数的作用。getchar()函数的原型为int getchar(void)，它从标准输入stdin中读取一个字符，并返回它的ASCII值。此处的while循环即可清除掉缓冲区中的所有内容。

4 运行演示

4.1 编译

我使用的操作系统是 **Windows 11 家庭中文版**，编译器是 **Visual Studio 2022**。把以上两个文件放入工程，接着右击项目名称，选择属性 → 链接器 → 输入 → 附加依赖项，添加 ws2_32.lib。然后分别编译生成 server.exe 与 client.exe。

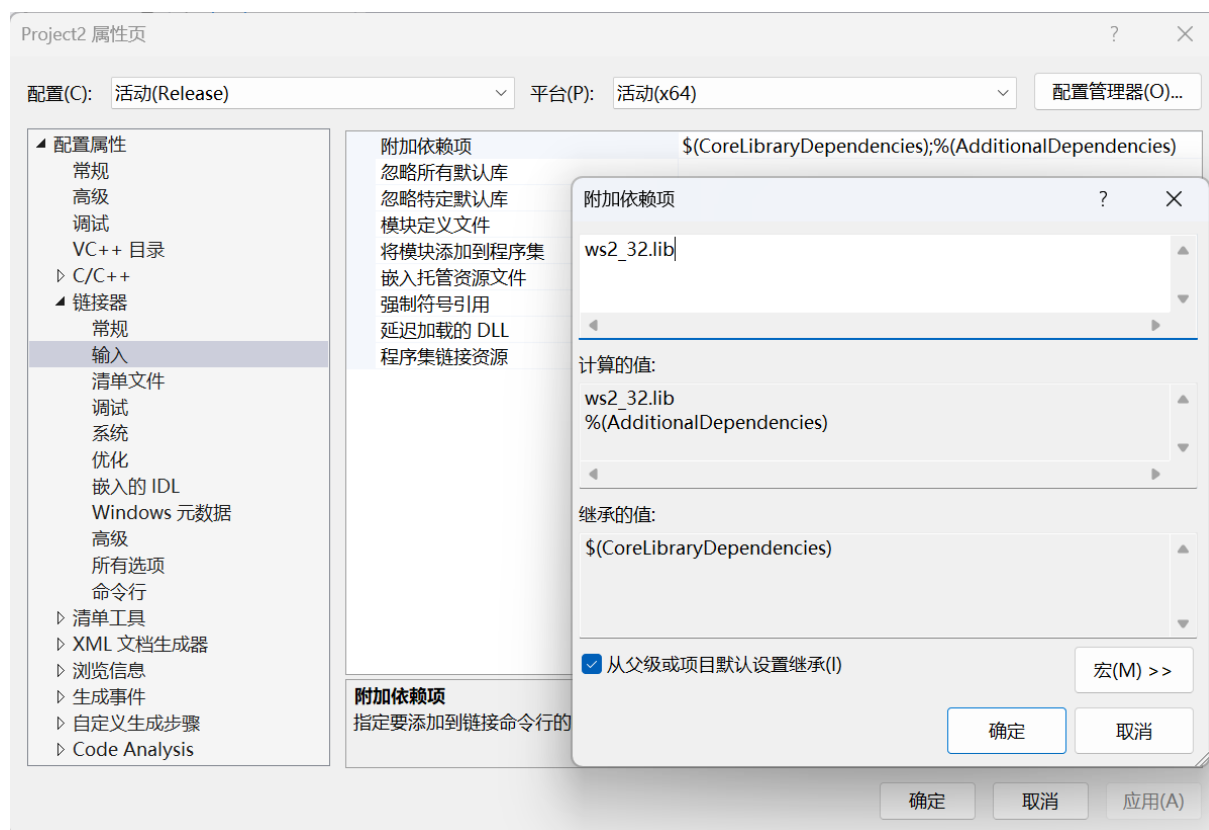


图 1: 链接 ws2_32.lib

 client.exe	2025/11/12 23:16	应用程序	15 KB
 server.exe	2025/11/12 23:53	应用程序	17 KB

图 2: 编译生成的 exe 可执行文件

4.2 运行效果

在自己的电脑上同时开启服务器端和多个客户端当然可以查看运行效果，但是为了更加真实模拟实际情况，我在我自己的电脑上开启服务器端和一个客户端，再在我舍友的电脑上开启一个客户端。为了能成功和舍友的电脑连接，须确保两台电脑在同一局域网下。我将两台电脑同时连接我的手机热点，执行ipconfig查看两台电脑的 IP 地址，如图3、4所示。

D:\南开批事情\第5学期-计算机网络\实验\实验1 利用流式套接字编写聊天程序>ipconfig

Windows IP 配置

无线局域网适配器 WLAN:

```
连接特定的 DNS 后缀 . . . . . :  
IPv6 地址 . . . . . : 2409:8902:60a7:133c:cdab:ad85:5c74:101f  
临时 IPv6 地址 . . . . . : 2409:8902:60a7:133c:7de9:9ec9:c92a:18cb  
本地链接 IPv6 地址 . . . . . : fe80::3eb:a5b:9e28:342d%22  
IPv4 地址 . . . . . : 192.168.193.235  
子网掩码 . . . . . : 255.255.255.0  
默认网关 . . . . . : fe80::7048:d0ff:fe78:7849%22  
192.168.193.182
```

图 3: 我的电脑 IP 地址 192.168.193.235，这也是服务器的 IP 地址

无线局域网适配器 WLAN:

```
连接特定的 DNS 后缀 . . . . . :  
IPv6 地址 . . . . . : 2409:8902:60a7:133c:8f75:2807:3647:87c4  
临时 IPv6 地址 . . . . . : 2409:8902:60a7:133c:3d86:2ed6:8343:1a79  
本地链接 IPv6 地址 . . . . . : fe80::6a6:8977:f5b0:ef6f%10  
IPv4 地址 . . . . . : 192.168.193.242  
子网掩码 . . . . . : 255.255.255.0  
默认网关 . . . . . : fe80::7048:d0ff:fe78:7849%10  
192.168.193.182
```

图 4: 舍友的电脑 IP 地址 192.168.193.242

然后关闭我的电脑的防火墙，防止舍友的访问被阻止。



图 5: 关闭防火墙

在 server.exe 所在路径下打开命令提示符，执行命令 `.\server.exe 8080` 启动服务器。此处自行分配 8080 端口，也可以分配其他端口。server.exe 和 client.c 的参数均通过命令传输：

server.c

```
int main(int argc, char* argv[]) {  
    // ...  
    int port = atoi(argv[1]);  
    // ...  
    server_addr.sin_port = htons(port);  
    // ...  
}
```

接着在我舍友的电脑上执行命令 `.\client.exe 192.168.193.235 8080` 启动客户端，输入用户名“张栩鸣”，此时该用户加入了聊天室，可以看到服务器端已经显示该用户的 IP 地址、端口和用户名。发送一条消息“你好！”，可以看到服务器端显示出了该消息。

然后在我的电脑上执行命令 `.\client.exe 192.168.193.235 8080` 启动客户端，输入用户名“蒋衲言”，此时一个新用户加入了聊天室，可以看到服务器端同样有显示，并且把 JOIN 消息发送给了其他用户。接着每个客户端都发送几条消息进行测试，然后分别退出聊天室，发现服务器端和每个客户端的运行均正常。它们各自的界面如图6、7、8所示。

```
C:\Users\Lenovo\Desktop>.\server.exe 8080  
南开大学《计算机网络》课程第1次实验作业  
一利用流式套接字编写聊天程序  
一蒋衲言 2313546  
这里是服务端 server  
  
聊天服务器启动在端口 8080  
等待客户端连接 ...  
新客户端连接, IP: 192.168.193.242, 端口: 65297, 当前用户数: 1  
用户 张栩鸣 加入了聊天室  
转发消息 [张栩鸣]: 你好!  
新客户端连接, IP: 192.168.193.235, 端口: 16422, 当前用户数: 2  
用户 蒋衲言 加入了聊天室  
转发消息 [蒋衲言]: 你也好!  
转发消息 [蒋衲言]: 哈哈哈哈哈  
转发消息 [张栩鸣]: 12345678  
客户端 张栩鸣 主动断开连接  
用户 张栩鸣 离开了聊天室  
客户端 蒋衲言 主动断开连接  
用户 蒋衲言 离开了聊天室
```

图 6: 服务器端

```

C:\Users\Lenovo\Desktop>.\client.exe 192.168.193.235 8080
南开大学《计算机网络》课程第1次实验作业
一利用流式套接字编写聊天程序
一蒋衲言 2313546
这里是客户端client

请输入你的用户名（最多30字节）： 蒋衲言
正在连接服务器 192.168.193.235:8080 ...
连接到聊天服务器成功！
输入 'exit' 退出聊天室
输入消息后按回车发送
=====
[蒋衲言]:
>>> 欢迎 蒋衲言 加入聊天室！
[蒋衲言]: 你也好！
[蒋衲言]: 哈哈哈哈哈
[蒋衲言]:
[张栩鸣]: 12345678
[蒋衲言]:
>>> 系统：用户 [张栩鸣] 离开了聊天室
[蒋衲言]: exit
退出聊天室...
已退出聊天室

```

图 7: 客户端“蒋衲言”

```

C:\Users\32762\Desktop>.\client.exe 192.168.193.235 8080
南开大学《计算机网络》课程第1次实验作业
一利用流式套接字编写聊天程序
一蒋衲言 2313546
这里是客户端client

请输入你的用户名（最多30字节）： 张栩鸣
正在连接服务器 192.168.193.235:8080 ...
连接到聊天服务器成功！
输入 'exit' 退出聊天室
输入消息后按回车发送
=====
[张栩鸣]:
>>> 欢迎 张栩鸣 加入聊天室！
[张栩鸣]: 你好！
[张栩鸣]:
>>> 系统：用户 [蒋衲言] 加入了聊天室
[张栩鸣]:
[蒋衲言]: 你也好！
[张栩鸣]:
[蒋衲言]: 哈哈哈哈哈
[张栩鸣]: 12345678
[张栩鸣]: exit
退出聊天室...
已退出聊天室

```

图 8: 客户端“张栩鸣”

4.3 观察是否有数据包的丢失

启动 Wireshark，选择 Loopback 接口，开始抓包。接着打开服务器端和客户端（使用回环地址 127.0.0.1、端口 8080 启动服务器端，这时在同一台电脑上启动客户端进行验证），客户端发送一条长消息：

计算机网络是计算机类专业的核心课程，在课程体系占有重要地位。课程以 TCP/IP 体系结构为核心、以互联网为实例，采用自顶向下的方法，介绍计算机网络层次划分的基本原则、各层次的基本原理和典型的网络协议、以及计算机网络技术的最新发展。通过课程的理论学习，学生能够系统掌握计算机网络的基本理论和基本方法，深入理解网络互联、路由转发、可靠传输、拥塞控制等网络核心概念，并能基于相关的数据模型对复杂系统工程问题进行分析。通过多种网络协议工作机制的学习和网络新技术的拓展，提高学生的自主学习能力，使学生能够综合运用相关机理及寻找可替代的解决方案，解决复杂的计算机网络工程问题。通过网络协议设计和编程训练，使学生能够掌握基本的网络编程方法，并具备网络协议分析和设计能力，以及解决复杂网络问题的能力，培养学生分析问题的能力和创新意识。通过计算机网络协议和计算机网络标准的学习，使学生了解计算机网络的标准体系，理解计算机网络标准对系统之间互操作的重要作用，了解标准制定对我国计算机网络技术发展的重要意义。

再随便发送一串数字“0123456789”。

停止抓包，在过滤栏输入 `tcp.port == 8080`，只显示聊天流量。右键任意 TCP 包 → Follow → TCP Stream，以 GBK 编码显示，发现消息完整，未发生丢失。并且所有 TCP 段均有对应 ACK 确认。因此，基于 TCP 的流式套接字聊天程序在实验环境中**未发生数据包丢失**，验证了 TCP 的**可靠传输**机制。

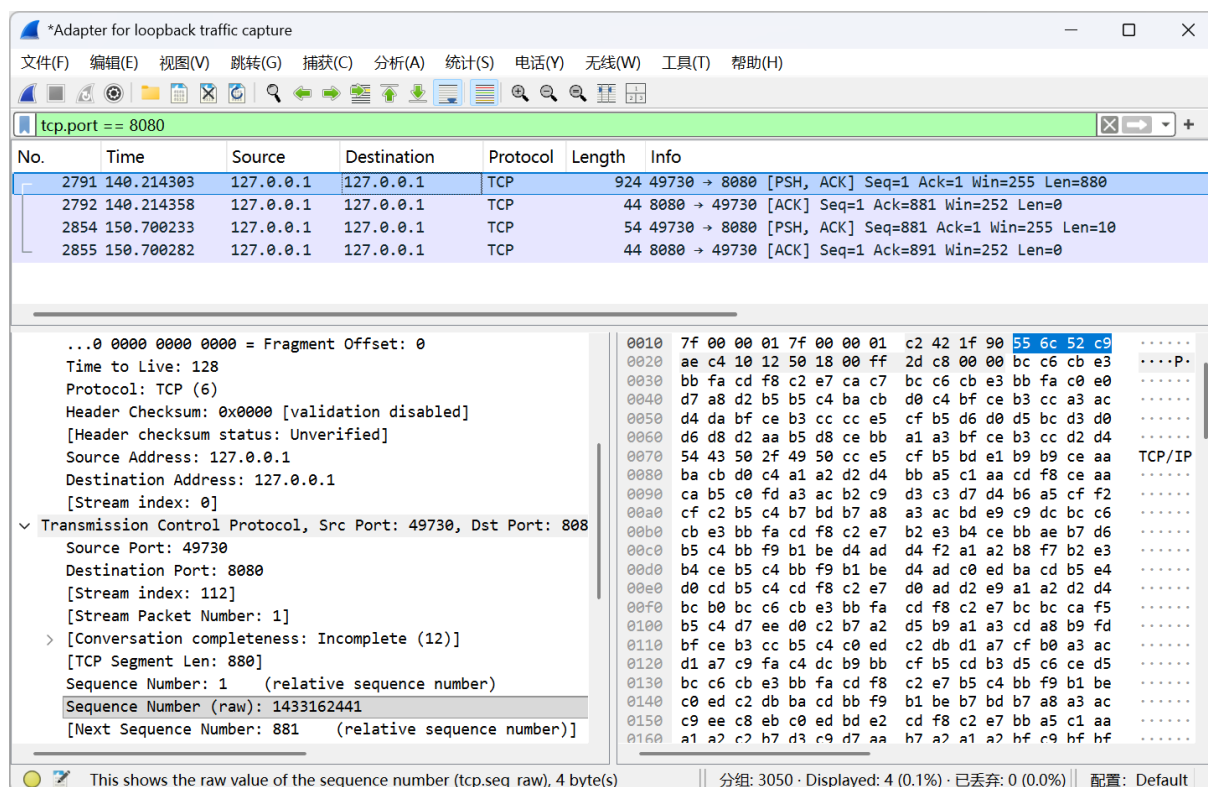


图 9: 过滤后的聊天数据包

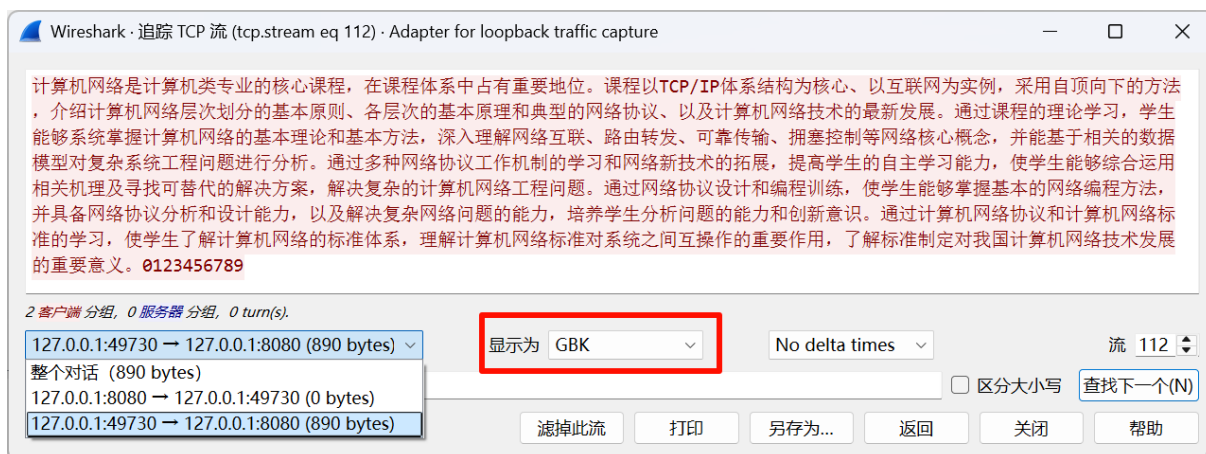


图 10: 完整聊天记录

本次实验至此全部完成。